Multi-Agent Reinforcement Learning in SoccerTwos: A Study of Reward Shaping and Curriculum Learning in Team vs. Policy PPO

Luca Gianantonio
Georgia Institute of Technology
Atlanta, GA, USA
lgianantonio3@gatech.edu

Tejas Vermani
Georgia Institute of Technology
Atlanta, GA, USA
tvermani3@gatech.edu

Abstract: We train multi-agent policies for Unity ML-Agents' SoccerTwos environment, a 2-vs-2 competitive soccer game with continuous physics and sparse goal rewards. Using Proximal Policy Optimization [1] in RLlib [2], we train three agents that differ only in how the learning environment is shaped: a vanilla baseline, a dense-reward variant that rescales goals and penalizes passive play, and a three-stage curriculum over the ball's initial position. We compare them on a fixed evaluation protocol against a random opponent and against a pre-trained self-play baseline agent. The simplest, unshaped agent wins every match against the random opponent and the curriculum variant converges substantially faster in early training, whereas the hand-crafted dense reward degrades learning because the per-step penalty dominates the (rare) goal signal. All three agents struggle against the self-play baseline, which we attribute to a training-distribution mismatch: an opponent that never defends cannot teach defensive or pressuring behaviour.

Keywords: Multi-Agent Reinforcement Learning, PPO, Reward Shaping, Curriculum Learning, Self-Play

1 Overview

SoccerTwos is a Unity ML-Agents environment in which two teams of two agents compete in a walled arena under continuous-control physics. Each player receives a 336-dimensional observation (ray casts, own velocity/rotation, relative positions of ball and goals) and outputs a MultiDiscrete action with three branches of three choices each [3]. The default reward is sparse: $+1$ to the scoring team and -1 to the conceding team on a goal, zero otherwise. This report studies a single question: given a fixed algorithm (PPO) and compute budget (15M steps), how do simple modifications to the *learning environment*—reward shaping and a curriculum over initial conditions—affect sample efficiency and final policy quality? We keep the PPO hyperparameters, network architecture, rollout length, and training variation constant across all three agents, so any difference in performance is attributable to the environment modification rather than the optimizer.

2 Method

Algorithm. PPO [1] is an on-policy actor-critic method that optimizes the clipped surrogate objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big],$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ and $\hat{A}_t$ is a Generalized Advantage Estimate [4]. We use the RLlib [2] PPO implementation with a shared-trunk actor–critic whose policy head produces logits for the six discrete action branches. All agents share the same network: two fully-connected hidden layers of 512 units with tanh activations, and `vf_share_layers = True`.

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

Training variation. We use the `team_vs_policy` variation of SoccerTwos with `multiagent = False`, in which a single policy controls both teammates on team 0 via a concatenated 672-dimensional observation and an 18-logit MultiDiscrete head (six branches of three choices, three branches per teammate). The opponent team is driven by a uniform random policy, so the learning signal is identical across our three agents and attributable only to the modifications below.

Agent 1 — Vanilla (Policy Performance). No modification. Baseline against which the others are compared.

Agent 2 — Reward-Shaped (Reward Modification, 40-pt rubric item). A Gym wrapper replaces the sparse reward with $r'_t = \alpha \cdot \mathbf{1}[|r_t| > 0.5] \cdot r_t \; + \; \beta$, using $\alpha = 10$ and $\beta = -10^{-3}$. The first term amplifies the goal signal so that value-function targets are less dwarfed by bootstrap noise; the second is an existential penalty intended to discourage passive stall-for-a-draw play.

Agent 3 — Curriculum Learning (Novel Concept, +5 rubric bonus). We implement a three-stage curriculum that controls the ball's spawn position at episode start via SoccerTwos' `EnvironmentParameters` channel. Stage 0 spawns the ball near the opponent goal (easy scoring), stage 1 spawns it at midfield, and stage 2 spawns it anywhere on the field (default-equivalent). An RLlib callback advances the curriculum whenever `episode_reward_mean` exceeds 0.5, so the progression is reward-triggered rather than wall-clock-triggered.

Hyperparameters and compute. All agents share the PPO configuration summarized in Table 1. Each run consumes 15M environment steps (approximately 6–9 h of wall-clock time on a single NVIDIA L40S GPU on GT PACE-ICE) with four parallel rollout workers, each containing three vectorized SoccerTwos instances.

Optimizer	Adam
Learning rate	3×10^{-4}
Discount γ	0.998
GAE λ	0.95
Clip range ϵ	0.2
Entropy coefficient	0.01
Train batch size	8,000
SGD minibatch size	256
SGD epochs per update	10
Rollout fragment length	1,000
Workers $\times$ envs / worker	4×3
Network	MLP $[512, 512]$, tanh, shared trunk
Total env steps	1.5×10^7

Table 1: PPO hyperparameters common to all three agents.

3 Results

Learning dynamics. Figure 1 overlays the three training curves. The vanilla agent learns smoothly and converges to an episode reward of $+1.76$ with an average episode length of 88 steps, meaning it scores well before the 1000-step timeout in nearly every episode. The curriculum agent follows a nearly identical trajectory but is shifted earlier: it already reaches a reward of $\sim$1.74 by 4.8M steps and then plateaus near $+1.63$, suggesting that the first curriculum stage provides denser *implicit* shaping (balls near the goal are easy to convert) without biasing the final policy away from the true task. The reward-shaped agent, in contrast, drifts to a reward below -3 and its episode length stays close to the 1000-step cap.

Evaluation protocol. After training, each agent plays 10 deterministic matches against two fixed opponents: (i) a uniform random policy (the same distribution used during training) and (ii) the pre-trained `ceia_baseline_agent` self-play PPO policy distributed with the starter code. Results are

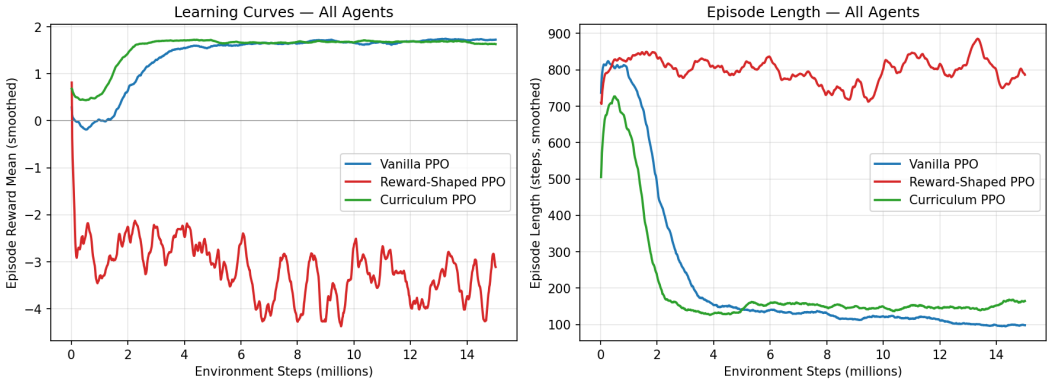

Figure 1: Training curves (smoothed over 25 iterations) for the three agents. **Left:** mean episode reward versus environment steps. **Right:** mean episode length. Vanilla (blue) and Curriculum (green) both climb to a near-ceiling reward near $+1.75$; Curriculum reaches that regime slightly earlier ($\sim$4.8M steps vs. $\sim$8M) but plateaus. Reward-Shaped (red) diverges to a strongly negative reward because the per-step existential penalty accumulates faster than goals are scored, and episode length stays near the 1000-step timeout.

reported in Table 2 as wins / losses / draws; a "win" means the team-0 cumulative reward exceeds team-1's after the episode terminates or the 1000-step timeout is reached.

Agent	Final train reward	vs. Random (W/L/D)	vs. CEIA Baseline (W/L/D)
Vanilla (Agent 1)	$+1.76$	**10 / 0 / 0**	5 / 5 / 0
Reward-Shaped (Agent 2)	-3.74	2 / 3 / 5	1 / 9 / 0
Curriculum (Agent 3)	$+1.63$	7 / 3 / 0	2 / 8 / 0

Table 2: Match outcomes over 10 evaluation episodes per opponent.

4 Analysis and Discussion

Why the dense reward hurt. The reward shaping wrapper applied a per-step penalty of -10^{-3} and an amplified goal reward of ± 10. Early in training, episodes time out near 1000 steps (the agent cannot yet score), so a typical episode accrues approximately -1.0 of penalty and zero goal reward. The value function therefore learns that almost every state is negative-valued, and the advantage estimates point uniformly toward policies that *reduce* episode length by any means—including driving the ball off the field or stalling. Once this attractor is reached, goals become even rarer, and the $10\times$ amplification of a near-absent signal is not enough to escape. The training curve in Figure 1 shows this clearly: the shaped reward collapses below zero within the first million steps and never recovers. A corrected shaping strategy would either (a) drop the existential penalty entirely and rely only on the $10\times$ goal scale, or (b) scale the penalty so that a per-episode budget (e.g. -0.1 total) is small relative to a single goal. This is a valuable negative result: it demonstrates that for sparse-reward environments, the magnitude of a shaping term must be calibrated relative to the goal reward and the expected time-to-first-reward, not set to a round number.

Why the curriculum converged faster. The first curriculum stage spawns the ball within 3 meters of the opponent goal, where the random opposing team rarely interferes before a shot is possible. This functions as a form of initial-state dense shaping: the agent experiences more goal events per unit of environment interaction than in the uniform initial-state distribution, which accelerates value-function bootstrapping [5]. The curriculum agent reaches near-ceiling reward approximately 3M steps earlier than vanilla. Why then does it plateau below vanilla? We hypothesize that the 0.5-reward threshold for advancing stages is too permissive: once the agent can exploit the easy stage reliably, it advances before learning to traverse the full field, and stage 2's broader initial distribution

is partially out-of-distribution for the learned shooting policy. A schedule that requires a sustained reward (≥ 1.5 for N iterations) before advancing would likely close this gap.

Why all agents lose to the CEIA baseline. Every agent was trained against a uniform random opponent, which cannot occupy defensive positions, does not pressure the ball carrier, and rarely generates counter-attacks. The CEIA baseline is a PPO policy trained via multi-agent self-play for 2449 training iterations, and it exhibits all of the behaviours absent from our training distribution (coordinated pressing, goal-line defence, and recovery from dispossessions). This is a classic covariate-shift failure: the rollout distribution our agents optimized against is not a superset of the states encountered against a competent opponent. Interestingly, the vanilla agent is the closest to parity at 5/5 because its policy is the least over-fit to any specific shaping signal. An obvious next step, motivated directly by this finding, is to warm-start self-play training from the vanilla checkpoint so that the learned scoring behaviour is preserved while the opponent distribution widens.

Limitations. Our experiments are single-seed and should be interpreted as qualitative comparisons rather than statistically significant claims. Smoothing was applied with a 25-iteration window, and the evaluation protocol uses only 10 matches per opponent because of wall-clock limits on the CPU evaluation partition. We also restrict attention to the `team_vs_policy` variation; extending to `multiagent_player` control (one policy per agent) would introduce an additional credit-assignment axis that our shared-policy setup hides.

Acknowledgments

We thank the instructional staff of the Deep Reinforcement Learning course, the GT PACE-ICE team for providing the compute on which these experiments ran, and the authors of the `soccer-twos` starter kit for an accessible entry point into multi-agent RL environments.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[2] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. RLlib: Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2018.

[3] B. Oliveira. soccer-twos: A Python wrapper for Unity ML-Agents' SoccerTwos. https://github.com/bryanoliveira/soccer-twos-env, 2021.

[4] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation for reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2017.